

Quantium Virtual Internship — Task 1

Retail Strategy and Analytics: Análisis exploratorio de datos (EDA) en Python

Este notebook es la traducción a Python del template en R Markdown (InsideSherpa_Task1_DraftSolutions).

Datasets requeridos :

- QVI_transaction_data.csv
- QVI_purchase_behaviour.csv

0. Cargar librerías

```
In [52]: %pip install pandas numpy matplotlib seaborn scipy
```

Requirement already satisfied: pandas in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (3.0.3)

Requirement already satisfied: numpy in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (2.4.6)

Requirement already satisfied: matplotlib in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (3.11.0)

Requirement already satisfied: seaborn in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (0.13.2)

Requirement already satisfied: scipy in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (1.17.1)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: tzdata in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from pandas) (2026.2)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cyclor>=0.10 in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from matplotlib) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from matplotlib) (1.5.0)

Requirement already satisfied: packaging>=20.0 in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from matplotlib) (25.0)

Requirement already satisfied: pillow>=9 in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from matplotlib) (12.3.0)

Requirement already satisfied: pyparsing>=3 in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from matplotlib) (3.3.2)

Requirement already satisfied: six>=1.5 in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 24.0 -> 26.1.2

[notice] To update, run: C:\Users\andre\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

In [53]: %pip install openpyxl

Requirement already satisfied: openpyxl in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (3.1.5)

Requirement already satisfied: et-xmlfile in c:\users\andre\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from openpyxl) (2.0.0)

Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 24.0 -> 26.1.2

[notice] To update, run: C:\Users\andre\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

In []:

In [54]: *# pandas: manipulación y análisis de datos tabulares (equivalente a data.table en R)*

```
import pandas as pd
```

numpy: operaciones numéricas de soporte (usado internamente por pandas y para cálculos)

```
import numpy as np
```

re: expresiones regulares, para limpiar texto (nombres de producto)

```
import re
```

Counter: para contar frecuencias de palabras de forma sencilla

```
from collections import Counter
```

matplotlib: motor base de gráficos

```
import matplotlib.pyplot as plt
```

seaborn: gráficos estadísticos más elaborados sobre matplotlib (barras agrupadas, etc.)

```
import seaborn as sns
```

scipy.stats: para la prueba de hipótesis (t-test) más adelante

```
from scipy import stats
```

Configuración visual: tamaño de fuente y estilo de fondo de los gráficos

```
sns.set_theme(style="whitegrid") # equivalente a theme_bw() + theme_set() en ggplot2
plt.rcParams["figure.figsize"] = (12, 6) # tamaño por defecto de las figuras
```

In [55]: `import sys`

```
print(sys.executable)
print(sys.version)
```

C:\Users\andre\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe
3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]

In [56]: `from pathlib import Path`

```
file_path = Path(r"C:\Users\andre\OneDrive\Documentos\M1D@5_777\7_Simulaciones de t")

print("¿Existe la carpeta?", file_path.exists())
print("¿Es un directorio?", file_path.is_dir())

print("\nContenido:")
```

```
for archivo in file_path.iterdir():  
    print(archivo.name)
```

```
¿Existe la carpeta? True  
¿Es un directorio? True
```

Contenido:

```
.git  
.venv  
InsideSherpa_Task1_DraftSolutions - Template.pdf  
Quantium_Task1_EDA_Python.html  
Quantium_Task1_EDA_Python.ipynb  
Quantium_Task1_EDA_Python.pdf  
QVI_data.csv  
QVI_purchase_behaviour.csv  
QVI_transaction_data.xlsx
```

In []:

1. Cargar los datasets

```
In [57]: # Ruta donde están guardados los archivos CSV; editala según tu entorno  
file_path = r"C:\Users\andre\OneDrive\Documentos\M1D@5_777\7_Simulaciones de trabaj  
  
# Leemos el archivo de transacciones y lo cargamos como DataFrame de pandas  
transaction_data = pd.read_excel(file_path + "QVI_transaction_data.xlsx")  
  
# Leemos el archivo de comportamiento de compra / segmentación de clientes  
customer_data = pd.read_csv(file_path + "QVI_purchase_behaviour.csv")
```

In []:

2. Exploración inicial de transaction_data

In [58]: `# .info() muestra tipos de dato por columna, cantidad de nulos y uso de memoria`
`# (equivalente a str() en R): nos deja ver si DATE está como número en vez de fecha`
`transaction_data.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   DATE                  264836 non-null int64
1   STORE_NBR             264836 non-null int64
2   LYLTY_CARD_NBR        264836 non-null int64
3   TXN_ID                264836 non-null int64
4   PROD_NBR              264836 non-null int64
5   PROD_NAME             264836 non-null str
6   PROD_QTY              264836 non-null int64
7   TOT_SALES             264836 non-null float64
dtypes: float64(1), int64(6), str(1)
memory usage: 16.2 MB
```

In [59]: `# .head() muestra las primeras 5 filas para inspeccionar visualmente el contenido`
`transaction_data.head()`

Out[59]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY
--	------	-----------	----------------	--------	----------	-----------	----------

0	43390	1	1000	1	5	Chip Natural Compny SeaSalt175g	2
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	3
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	4
3	43329	2	2373	974	69	Smiths Chip Thinly S/ Cream&Onion 175g	5
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Ulpno Chili 150g	6

In [60]: `# .describe() calcula estadísticas descriptivas (media, min, max, percentiles)`
`# de las columnas numéricas; ayuda a detectar outliers desde ya`
`transaction_data.describe()`

Out[60]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PR
count	264836.000000	264836.00000	2.648360e+05	2.648360e+05	264836.000000	264836
mean	43464.036260	135.08011	1.355495e+05	1.351583e+05	56.583157	
std	105.389282	76.78418	8.057998e+04	7.813303e+04	32.826638	
min	43282.000000	1.00000	1.000000e+03	1.000000e+00	1.000000	
25%	43373.000000	70.00000	7.002100e+04	6.760150e+04	28.000000	
50%	43464.000000	130.00000	1.303575e+05	1.351375e+05	56.000000	
75%	43555.000000	203.00000	2.030942e+05	2.027012e+05	85.000000	
max	43646.000000	272.00000	2.373711e+06	2.415841e+06	114.000000	20

3. Convertir la columna DATE a formato fecha

La columna DATE viene como un número entero (formato serial de Excel/CSV), donde el día 0 corresponde al 30 de diciembre de 1899. Hay que convertirla a un tipo de dato `datetime` para poder graficarla y agruparla correctamente.

```
In [61]: # unit="D" indica que el número representa una cantidad de días
# origin="1899-12-30" es el punto de partida que usan Excel y CSV para fechas serial
transaction_data["DATE"] = pd.to_datetime(
    transaction_data["DATE"], unit="D", origin="1899-12-30"
)

# Verificamos que la conversión fue exitosa mostrando el nuevo tipo de dato
transaction_data["DATE"].head()
```

```
Out[61]: 0    2018-10-17
1    2019-05-14
2    2019-05-20
3    2018-08-17
4    2018-08-18
Name: DATE, dtype: datetime64[s]
```

4. Examinar PROD_NAME

Queremos confirmar que todos los productos son efectivamente "papas fritas" (chips) y no otra categoría que se haya colado en el dataset (por ejemplo, salsas).

```
In [62]: # .value_counts() cuenta cuántas veces aparece cada nombre de producto distinto
# .head(20) limita la salida a los 20 productos más frecuentes, para no saturar la
transaction_data["PROD_NAME"].value_counts().head(20)
```

```
Out[62]: PROD_NAME
Kettle Mozzarella Basil & Pesto 175g 3304
Kettle Tortilla ChpsHny&Jlpno Chili 150g 3296
Cobs Popd Swt/Chlli &Sr/Cream Chips 110g 3269
Tyrrells Crisps Ched & Chives 165g 3268
Cobs Popd Sea Salt Chips 110g 3265
Kettle 135g Swt Pot Sea Salt 3257
Tostitos Splash Of Lime 175g 3252
Infuzions Thai SweetChili PotatoMix 110g 3242
Smiths Crnkle Chip Orgnl Big Bag 380g 3233
Thins Potato Chips Hot & Spicy 175g 3229
Kettle Sensations Camembert & Fig 150g 3219
Doritos Corn Chips Cheese Supreme 170g 3217
Pringles Barbeque 134g 3210
Doritos Corn Chip Mexican Jalapeno 150g 3204
Kettle Sweet Chilli And Sour Cream 175g 3200
Smiths Crinkle Chips Salt & Vinegar 330g 3197
Thins Chips Light& Tangy 175g 3188
Dorito Corn Chp Supreme 380g 3185
Pringles Sweet&Spcy BBQ 134g 3177
Infuzions BBQ Rib Prawn Crackers 110g 3174
Name: count, dtype: int64
```

```
In [63]: # Unimos todos Los nombres de producto ÚNICOS en un solo string separado por espaci
# .unique() evita contar La misma palabra muchas veces solo porque el producto se r
all_names = " ".join(transaction_data["PROD_NAME"].unique())

# .split() separa el string gigante en una lista de palabras individuales (por espa
words = all_names.split()

# Mostramos cuántas palabras en total se extrajeron, solo para verificar
len(words)
```

```
Out[63]: 589
```

```
In [64]: # re.sub(r"^[A-Za-z]", "", w) elimina cualquier caracter que NO sea una letra
# (quita dígitos como "175g" -> "g", símbolos como "&", etc.)
clean_words = [re.sub(r"^[A-Za-z]", "", w) for w in words]

# Filtramos Los strings que quedaron vacíos después de La limpieza (por ejemplo "17
clean_words = [w for w in clean_words if w != ""]

# Counter cuenta cuántas veces aparece cada palabra limpia y .most_common() Las ord
# de mayor a menor frecuencia
word_freq = Counter(clean_words).most_common()

# Convertimos el resultado a un DataFrame para visualizarlo como tabla ordenada
word_freq_df = pd.DataFrame(word_freq, columns=["word", "count"])

# Mostramos Las 20 palabras más frecuentes: aquí es donde detectaríamos "Salsa"
# si hubiera productos que no son chips
word_freq_df.head(20)
```

Out[64]:

	word	count
0	g	105
1	Chips	21
2	Smiths	16
3	Crinkle	14
4	Cut	14
5	Kettle	13
6	Cheese	12
7	Salt	12
8	Original	10
9	Chip	9
10	Salsa	9
11	Doritos	9
12	Corn	8
13	Pringles	8
14	RRD	8
15	Chicken	7
16	WW	7
17	Chilli	6
18	Sour	6
19	Sea	6

5. Eliminar productos de salsa (no pertenecen a la categoría chips)

```
In [65]: # .str.lower() convierte el nombre del producto a minúsculas para comparar sin
# importar mayúsculas/minúsculas
# .str.contains("salsa") devuelve True/False si la palabra "salsa" aparece en el no
is_salsa = transaction_data["PROD_NAME"].str.lower().str.contains("salsa")

# El operador ~ invierte la máscara booleana (True se vuelve False y viceversa)
# Así nos quedamos solo con las filas donde is_salsa es False (no son salsa)
transaction_data = transaction_data[~is_salsa]

# Confirmamos cuántas filas quedaron después de filtrar
transaction_data.shape
```


Out[65]: (246742, 8)

6. Resumen estadístico y detección de outliers

Revisamos nulos y valores extremos en las columnas clave antes de seguir.

```
In [66]: # .isna() marca True donde hay valores nulos; .sum() cuenta cuántos hay por columna
# Si todos los valores son 0, significa que no hay datos faltantes
transaction_data.isna().sum()
```

```
Out[66]: DATE                0
STORE_NBR                  0
LYLTY_CARD_NBR            0
TXN_ID                    0
PROD_NBR                  0
PROD_NAME                 0
PROD_QTY                  0
TOT_SALES                 0
dtype: int64
```

```
In [67]: # Volvemos a correr describe() ya con los datos filtrados, para ver si PROD_QTY
# (cantidad de productos por transacción) tiene un máximo sospechosamente alto
transaction_data.describe()
```

```
Out[67]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROI
count	246742	246742.000000	2.467420e+05	2.467420e+05	246742.000000	246742.0
mean	2018-12-30 01:19:01	135.051098	1.355310e+05	1.351311e+05	56.351789	1.9
min	2018-07-01 00:00:00	1.000000	1.000000e+03	1.000000e+00	1.000000	1.0
25%	2018-09-30 00:00:00	70.000000	7.001500e+04	6.756925e+04	26.000000	2.0
50%	2018-12-30 00:00:00	130.000000	1.303670e+05	1.351830e+05	53.000000	2.0
75%	2019-03-31 00:00:00	203.000000	2.030840e+05	2.026538e+05	87.000000	2.0
max	2019-06-30 00:00:00	272.000000	2.373711e+06	2.415841e+06	114.000000	200.0
std	NaN	76.787096	8.071528e+04	7.814772e+04	33.695428	0.6

```
In [68]: # Filtramos las transacciones donde se compraron exactamente 200 unidades,
# el valor atípico (outlier) que detectamos en el describe() anterior
outlier_transactions = transaction_data[transaction_data["PROD_QTY"] == 200]

# Mostramos esas transacciones para inspeccionarlas manualmente
outlier_transactions
```

Out[68]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PRC
69762	2018-08-19	226	226000	226201	4	Dorito Corn Chp Supreme 380g	
69763	2019-05-20	226	226000	226210	4	Dorito Corn Chp Supreme 380g	

```
In [69]: # Extraemos el/Los número(s) de tarjeta de Lealtad (LYLTY_CARD_NBR) involucrados
# en la transacción atípica, para revisar el historial completo de ese cliente
loyalty_card_outlier = outlier_transactions["LYLTY_CARD_NBR"].unique()

# .isin() filtra todas las filas donde el número de tarjeta esté en la lista anterior
# Esto nos muestra TODAS las compras hechas por ese cliente en el período analizado
transaction_data[transaction_data["LYLTY_CARD_NBR"].isin(loyalty_card_outlier)]
```

Out[69]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PRC
69762	2018-08-19	226	226000	226201	4	Dorito Corn Chp Supreme 380g	
69763	2019-05-20	226	226000	226210	4	Dorito Corn Chp Supreme 380g	

```
In [70]: # Como el cliente solo tiene esas dos transacciones extremas en todo el año,
# probablemente no es un consumidor minorista normal (podría ser compra comercial)
# Lo excluimos del análisis usando ~ (negación) sobre isin()
transaction_data = transaction_data[
    ~transaction_data["LYLTY_CARD_NBR"].isin(loyalty_card_outlier)
]

# Volvemos a describir los datos para confirmar que el outlier ya no aparece
transaction_data.describe()
```

Out[70]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROI
count	246740	246740.000000	2.467400e+05	2.467400e+05	246740.000000	246740.0
mean	2018-12-30 01:18:58	135.050361	1.355303e+05	1.351304e+05	56.352213	1.9
min	2018-07-01 00:00:00	1.000000	1.000000e+03	1.000000e+00	1.000000	1.0
25%	2018-09-30 00:00:00	70.000000	7.001500e+04	6.756875e+04	26.000000	2.0
50%	2018-12-30 00:00:00	130.000000	1.303670e+05	1.351815e+05	53.000000	2.0
75%	2019-03-31 00:00:00	203.000000	2.030832e+05	2.026522e+05	87.000000	2.0
max	2019-06-30 00:00:00	272.000000	2.373711e+06	2.415841e+06	114.000000	5.0
std	NaN	76.786971	8.071520e+04	7.814760e+04	33.695235	0.3

7. Conteo de transacciones por fecha (buscar días faltantes)

El dataset debería cubrir 365 días (1 jul 2018 - 30 jun 2019). Si el conteo de fechas únicas da menos de 365, significa que falta al menos un día de datos.

```
In [71]: # .groupby("DATE") agrupa las filas por fecha
# .size() cuenta cuántas transacciones (filas) hay en cada fecha
# .reset_index(name="N") convierte el resultado agrupado de nuevo en un DataFrame,
# nombrando la columna de conteo como "N"
tx_by_day = transaction_data.groupby("DATE").size().reset_index(name="N")

# len() nos dice cuántas fechas distintas hay en total; si es 364 en vez de 365,
# confirma que falta un día
len(tx_by_day)
```

Out[71]: 364

```
In [72]: # Creamos un rango de fechas completo, día por día, cubriendo TODO el período
# esperado (1 jul 2018 a 30 jun 2019), sin importar si hay datos ese día o no
full_dates = pd.DataFrame({
    "DATE": pd.date_range(start="2018-07-01", end="2019-06-30", freq="D")
})

# Unimos (left join) el rango completo de fechas con el conteo real de transacciones
# how="left" asegura que se mantengan TODAS las fechas del rango, aunque no tengan
tx_by_day = full_dates.merge(tx_by_day, on="DATE", how="left")

# Los días sin transacciones quedan como NaN después del merge; Los reemplazamos po
```

```
tx_by_day["N"] = tx_by_day["N"].fillna(0)

# Mostramos las primeras filas para confirmar que ahora sí hay 365 fechas
tx_by_day.shape
```

Out[72]: (365, 2)

```
In [73]: # Creamos la figura y los ejes con un tamaño específico (ancho=12, alto=5 pulgadas)
fig, ax = plt.subplots(figsize=(12, 5))

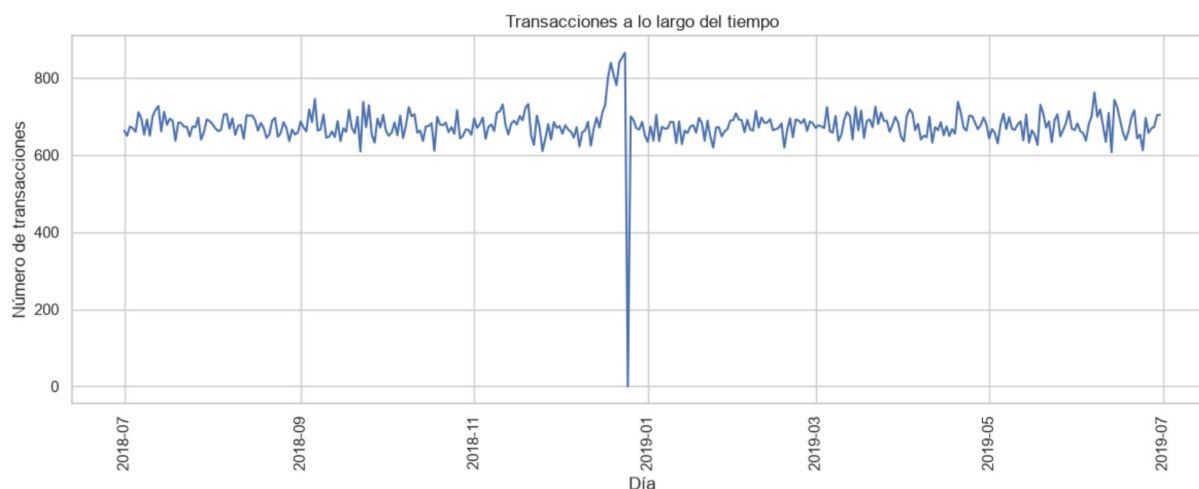
# Graficamos una línea con el eje X = fecha y el eje Y = número de transacciones
ax.plot(tx_by_day["DATE"], tx_by_day["N"])

# Etiquetas de los ejes, en español, para que el gráfico sea autoexplicativo
ax.set_xlabel("Día")
ax.set_ylabel("Número de transacciones")
ax.set_title("Transacciones a lo largo del tiempo")

# Rotamos las etiquetas del eje X 90 grados para que no se superpongan
plt.xticks(rotation=90)

# Ajusta automáticamente los márgenes para que no se corten las etiquetas
plt.tight_layout()

# Muestra el gráfico en el notebook
plt.show()
```



```
In [74]: # Filtramos tx_by_day para quedarnos solo con el mes de diciembre de 2018,
# donde vimos un pico y una caída en el gráfico anterior
december = tx_by_day[
    (tx_by_day["DATE"] >= "2018-12-01") & (tx_by_day["DATE"] <= "2018-12-31")
]

# Creamos el gráfico de línea con marcadores en cada punto (marker="o") para ver
# claramente cada día individual de diciembre
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(december["DATE"], december["N"], marker="o")

ax.set_xlabel("Día de diciembre")
ax.set_ylabel("Número de transacciones")
```



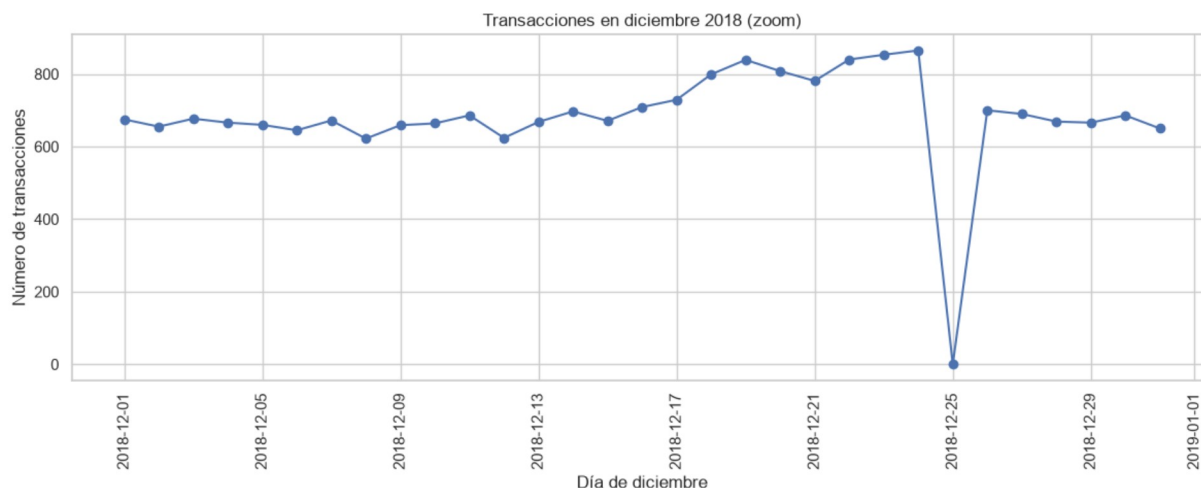
```
ax.set_title("Transacciones en diciembre 2018 (zoom)")
```

```
plt.xticks(rotation=90)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Se espera ver un aumento previo a Navidad y una caída a 0 el 25 de diciembre  
# (día en que las tiendas están cerradas)
```



8. Crear la variable PACK_SIZE

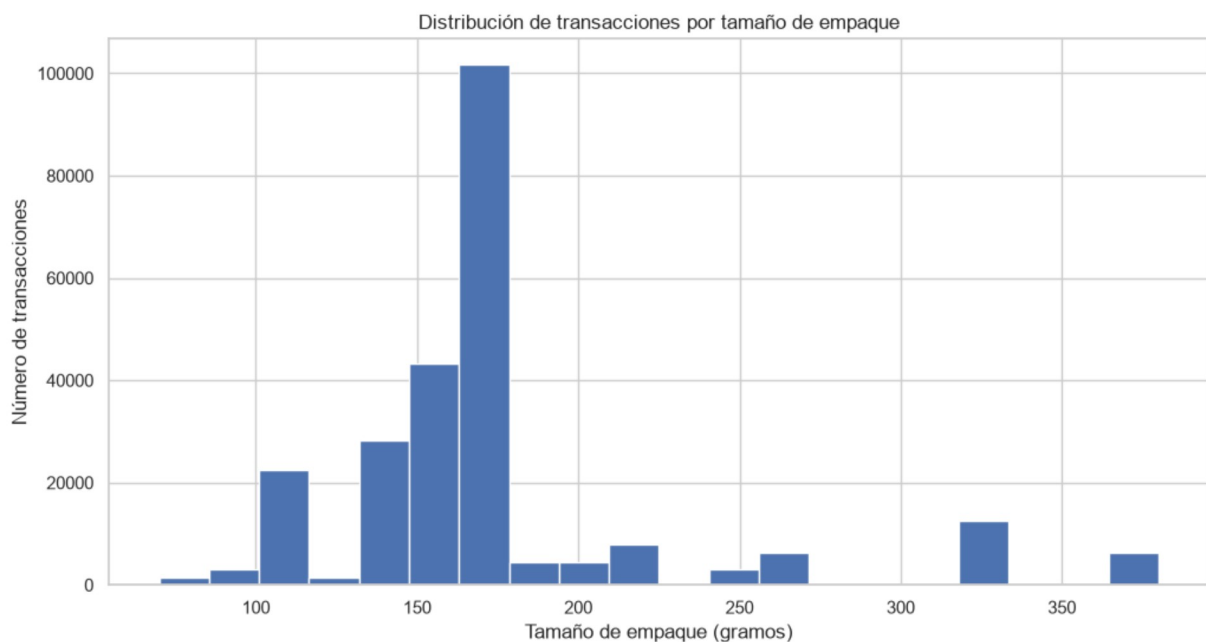
Extraemos el tamaño del empaque (en gramos) directamente del texto en `PROD_NAME`, ya que el número siempre aparece junto a la "g" de gramos (ej. "175g").

```
In [75]: # .str.extract(r"(\d+)") busca el primer grupo de uno o más dígitos consecutivos  
# en cada nombre de producto y lo devuelve como texto  
# .astype(int) convierte ese texto extraído en un número entero  
transaction_data["PACK_SIZE"] = (  
    transaction_data["PROD_NAME"].str.extract(r"(\d+)").astype(int)  
)  
  
# .value_counts() cuenta cuántas transacciones hay por cada tamaño de empaque  
# .sort_index() ordena el resultado de menor a mayor tamaño (no por frecuencia)  
transaction_data["PACK_SIZE"].value_counts().sort_index()
```

```
Out[75]: PACK_SIZE
70      1507
90      3008
110     22387
125     1454
134     25102
135      3257
150     40203
160      2970
165     15297
170     19983
175     66390
180      1468
190      2995
200     4473
210     6272
220     1564
250     3169
270     6285
330     12540
380      6416
Name: count, dtype: int64
```

```
In [76]: # Histograma: cada barra representa un rango (bin) de tamaños de empaque
# y su altura indica cuántas transacciones caen en ese rango
transaction_data["PACK_SIZE"].hist(bins=20)

plt.xlabel("Tamaño de empaque (gramos)")
plt.ylabel("Número de transacciones")
plt.title("Distribución de transacciones por tamaño de empaque")
plt.show()
```



9. Crear la variable BRAND

La marca suele ser la primera palabra del nombre del producto.

```
In [77]: # .str.split() separa el nombre del producto en una lista de palabras
# .str[0] toma la primera palabra de cada lista (asumimos que ahí está la marca)
# .str.upper() estandariza todo a mayúsculas para evitar duplicados por casing
transaction_data["BRAND"] = (
    transaction_data["PROD_NAME"].str.split().str[0].str.upper()
)

# Revisamos las marcas resultantes y cuántas transacciones tiene cada una
transaction_data["BRAND"].value_counts()
```

```
Out[77]: BRAND
KETTLE      41288
SMITHS      27390
PRINGLES    25102
DORITOS     22041
THINS       14075
RRD         11894
INFUZIONI   11057
WW          10320
COBS        9693
TOSTITOS    9471
TWISTIES    9454
TYRRELLS    6442
GRAIN       6272
NATURAL     6050
CHEEZELS    4603
CCS         4551
RED         4427
DORITO      3183
INFZNS      3144
SMITH       2963
CHEETOS     2927
SNBTS       1576
BURGER      1564
WOOLWORTHS  1516
GRNWVES     1468
SUNBITES    1432
NCC         1419
FRENCH      1418
Name: count, dtype: int64
```

```
In [78]: # Diccionario de corrección: distintas abreviaciones/errores de tipeo que en
# realidad corresponden a la misma marca real (se identifican revisando la lista an
brand_fix = {
    "RED": "RRD",          # "RED" es una abreviación de Red Rock Deli (RRD)
    "SNBTS": "SUNBITES",   # abreviación de Sunbites
    "INFZNS": "INFUZIONI", # abreviación de Infuzioni
    "WW": "WOOLWORTHS",    # abreviación de Woolworths
    "SMITH": "SMITHS",     # variación singular/plural de Smiths
    "NCC": "NATURAL",      # abreviación de Natural Chip Co
    "DORITO": "DORITOS",   # variación singular/plural de Doritos
    "GRAIN": "GRNWVES",    # abreviación de Grain Waves
}
```

```
# .replace() sustituye cada marca mal escrita por su versión estandarizada,
# usando el diccionario anterior como mapa de reemplazo
transaction_data["BRAND"] = transaction_data["BRAND"].replace(brand_fix)

# Verificamos que las marcas quedaron consolidadas correctamente
transaction_data["BRAND"].value_counts()
```

```
Out[78]: BRAND
KETTLE      41288
SMITHS      30353
DORITOS     25224
PRINGLES    25102
RRD         16321
INFUZIONI   14201
THINS       14075
WOOLWORTHS  11836
COBS        9693
TOSTITOS    9471
TWISTIES    9454
GRNWVES     7740
NATURAL     7469
TYRRELLS    6442
CHEEZELS    4603
CCS         4551
SUNBITES    3008
CHEETOS     2927
BURGER      1564
FRENCH      1418
Name: count, dtype: int64
```

10. Explorar customer_data

```
In [79]: # .info() para ver tipos de dato y nulos en el dataset de clientes
customer_data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   LYLTY_CARD_NBR   72637 non-null  int64
1   LIFESTAGE        72637 non-null  str
2   PREMIUM_CUSTOMER 72637 non-null  str
dtypes: int64(1), str(2)
memory usage: 1.7 MB
```

```
In [80]: # .describe(include="all") incluye también columnas de texto/categorías,
# mostrando frecuencias de las categorías más comunes (LIFESTAGE, PREMIUM_CUSTOMER)
customer_data.describe(include="all")
```



```
Out[80]:
```

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
count	7.263700e+04	72637	72637
unique	NaN	7	3
top	NaN	RETIREEES	Mainstream
freq	NaN	14805	29245
mean	1.361859e+05	NaN	NaN
std	8.989293e+04	NaN	NaN
min	1.000000e+03	NaN	NaN
25%	6.620200e+04	NaN	NaN
50%	1.340400e+05	NaN	NaN
75%	2.033750e+05	NaN	NaN
max	2.373711e+06	NaN	NaN

11. Unir (merge) transacciones con datos de cliente

```
In [81]: # merge() combina ambos DataFrames usando la columna en común "LYLTY_CARD_NBR"
# how="left" mantiene TODAS las filas de transaction_data, y les agrega la
# información de customer_data cuando encuentra una coincidencia
data = transaction_data.merge(customer_data, on="LYLTY_CARD_NBR", how="left")

# assert lanza un error si la condición es falsa: confirmamos que el merge NO
# generó filas adicionales (lo cual indicaría duplicados en customer_data)
assert len(data) == len(transaction_data), "El merge generó duplicados inesperados"

# Si no hay error, mostramos las dimensiones finales del dataset combinado
data.shape
```

```
Out[81]: (246740, 12)
```

```
In [82]: # Revisamos si alguna transacción quedó sin datos de cliente después del merge
# (es decir, si hay algún LIFESTAGE o PREMIUM_CUSTOMER nulo)
data[["LIFESTAGE", "PREMIUM_CUSTOMER"]].isna().sum()

# Si el resultado es 0 en ambas columnas, todos los clientes fueron encontrados
```

```
Out[82]: LIFESTAGE      0
PREMIUM_CUSTOMER    0
dtype: int64
```

```
In [83]: # Guardamos el dataset limpio y combinado en un nuevo archivo CSV,
# útil como punto de partida para el Task 2 del internship
data.to_csv(file_path + "QVI_data.csv", index=False)
```

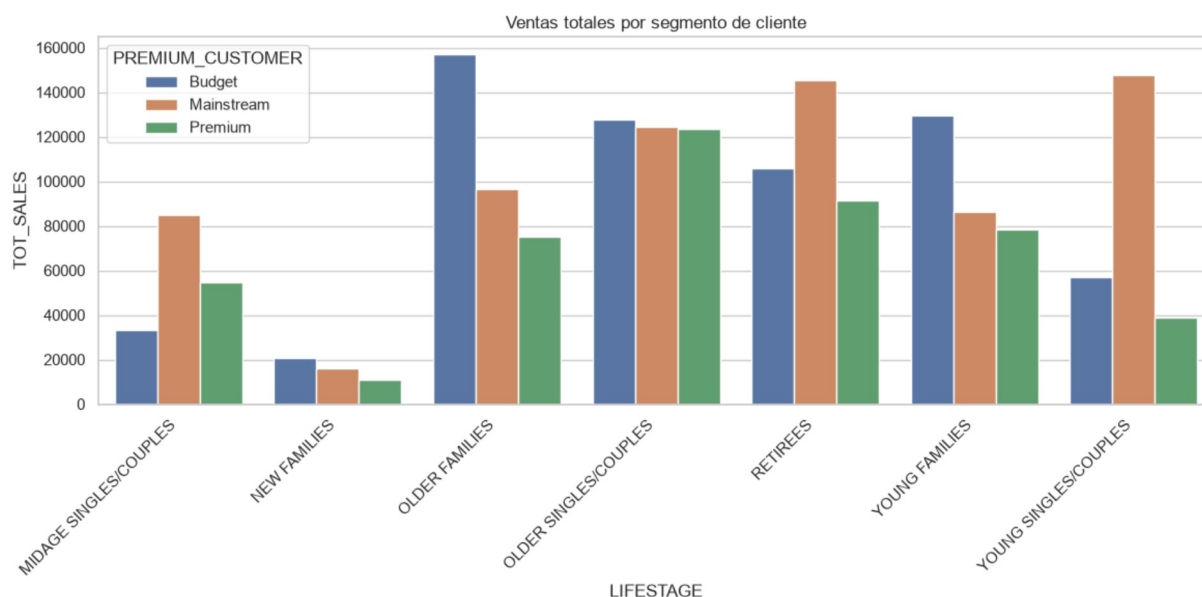
12. Análisis por segmento de cliente (LIFESTAGE x PREMIUM_CUSTOMER)

Ahora calculamos las métricas clave para entender qué segmentos impulsan más las ventas de chips.

```
In [84]: # Agrupamos por Las dos dimensiones de segmentación y sumamos TOT_SALES
# (ventas totales) dentro de cada combinación de grupo
sales_by_segment = (
    data.groupby(["LIFESTAGE", "PREMIUM_CUSTOMER"])[ "TOT_SALES" ]
    .sum()                                     # suma las ventas dentro de cada grupo
    .reset_index()                           # convierte el resultado agrupado en DataFrame
)

# Gráfico de barras agrupadas: eje X = etapa de vida, color = tipo de comprador,
# altura de barra = ventas totales
plt.figure(figsize=(12, 6))
sns.barplot(data=sales_by_segment, x="LIFESTAGE", y="TOT_SALES", hue="PREMIUM_CUSTO

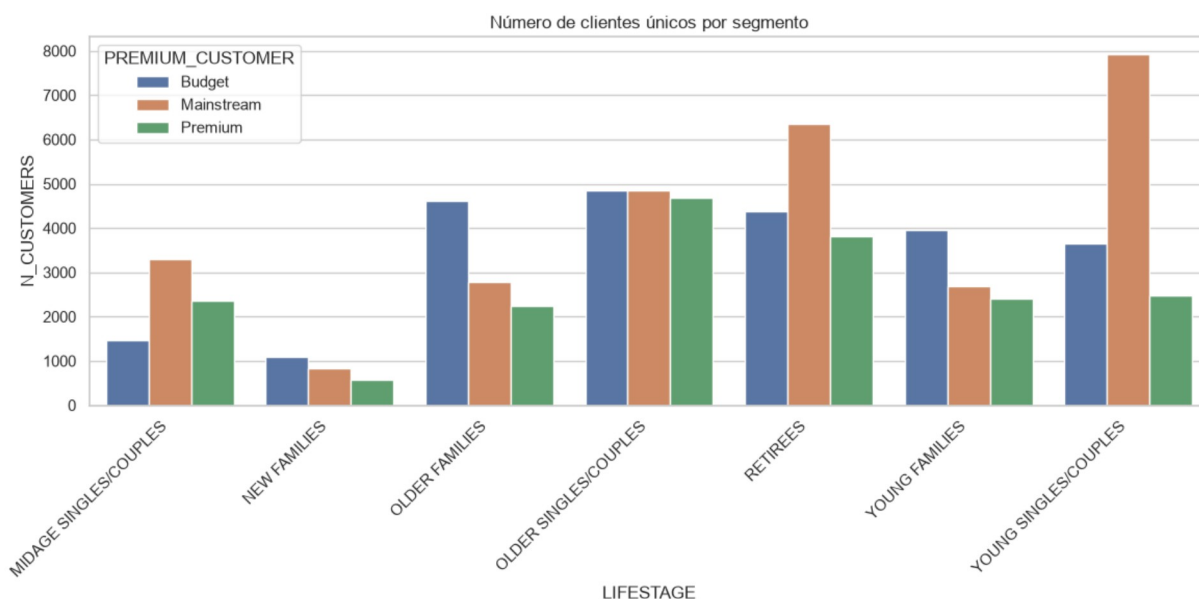
plt.xticks(rotation=45, ha="right") # rota etiquetas para que no se superpongan
plt.title("Ventas totales por segmento de cliente")
plt.tight_layout()
plt.show()
```



```
In [85]: # unique() cuenta cuántas tarjetas de Lealtad DISTINTAS hay en cada grupo,
# es decir, cuántos clientes únicos compran en cada segmento
customers_by_segment = (
    data.groupby(["LIFESTAGE", "PREMIUM_CUSTOMER"])[ "LYLTY_CARD_NBR" ]
    .nunique()
    .reset_index(name="N_CUSTOMERS")
)

plt.figure(figsize=(12, 6))
sns.barplot(data=customers_by_segment, x="LIFESTAGE", y="N_CUSTOMERS", hue="PREMIUM
```

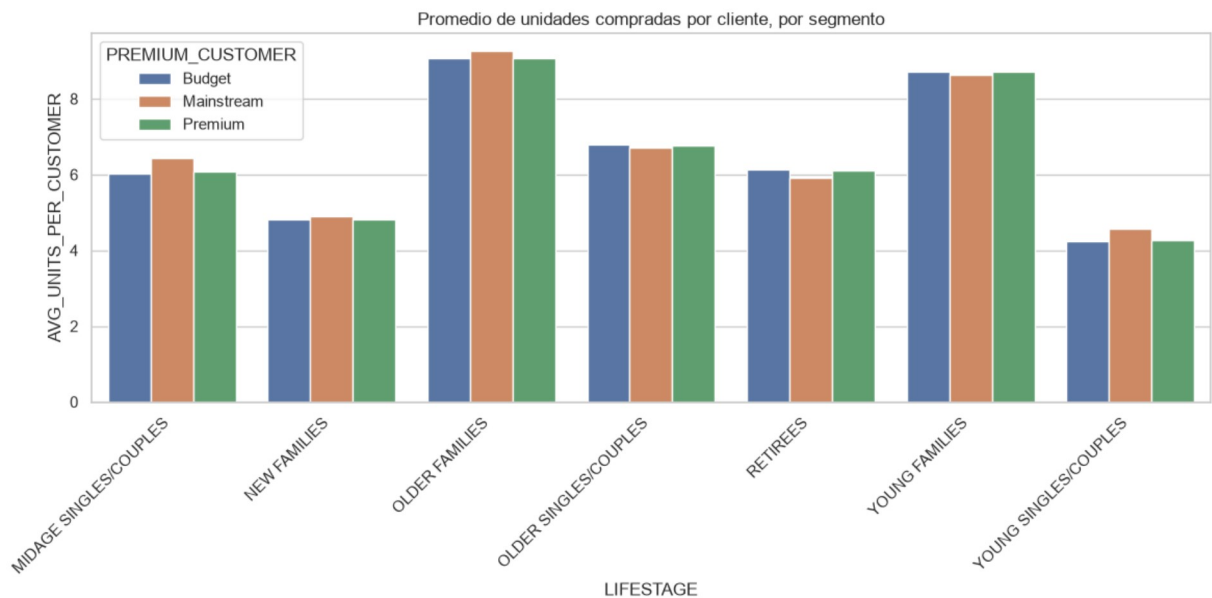
```
plt.xticks(rotation=45, ha="right")
plt.title("Número de clientes únicos por segmento")
plt.tight_layout()
plt.show()
```



```
In [86]: # Función auxiliar: para cada grupo, suma la cantidad total de productos comprados
# (PROD_QTY) y la divide entre el número de clientes únicos del grupo
def avg_units_per_customer(group):
    total_units = group["PROD_QTY"].sum()           # unidades totales del grup
    unique_customers = group["LYLTY_CARD_NBR"].nunique() # clientes únicos del grup
    return total_units / unique_customers           # promedio de unidades por

# .apply() ejecuta la función anterior sobre cada grupo definido por groupby()
units_per_customer = (
    data.groupby(["LIFESTAGE", "PREMIUM_CUSTOMER"])
    .apply(avg_units_per_customer)
    .reset_index(name="AVG_UNITS_PER_CUSTOMER")
)

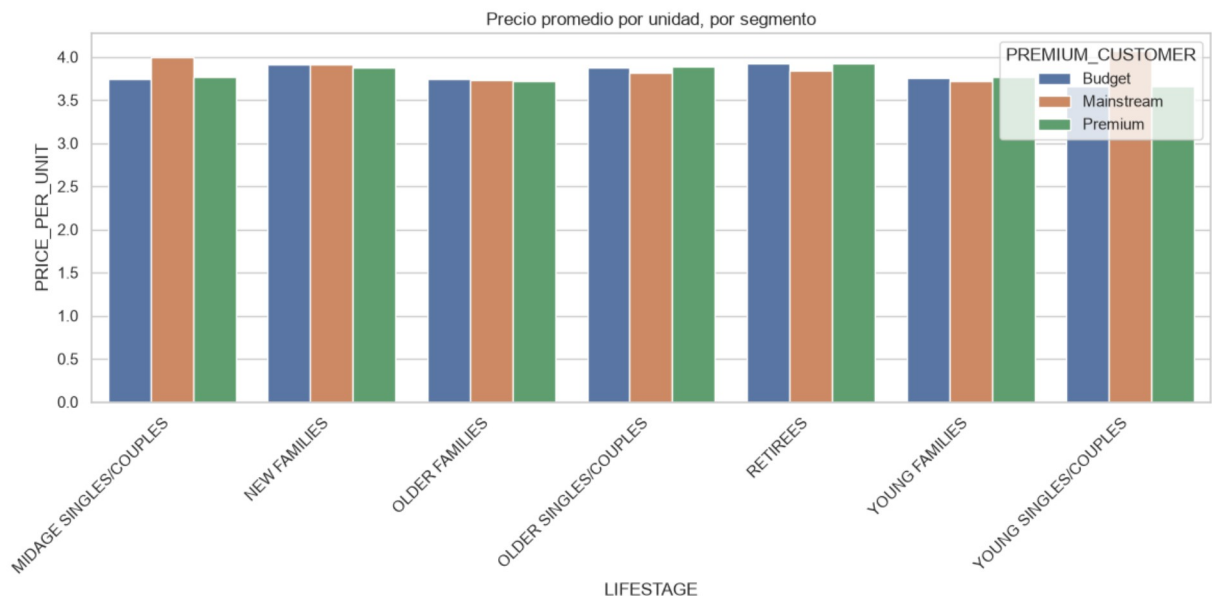
plt.figure(figsize=(12, 6))
sns.barplot(data=units_per_customer, x="LIFESTAGE", y="AVG_UNITS_PER_CUSTOMER", hue
plt.xticks(rotation=45, ha="right")
plt.title("Promedio de unidades compradas por cliente, por segmento")
plt.tight_layout()
plt.show()
```



```
In [87]: # Calculamos el precio pagado por unidad en cada transacción:
# ventas totales de la transacción / cantidad de unidades compradas
data["PRICE_PER_UNIT"] = data["TOT_SALES"] / data["PROD_QTY"]

# Promediamos ese precio por unidad dentro de cada segmento de cliente
avg_price = (
    data.groupby(["LIFESTAGE", "PREMIUM_CUSTOMER"])["PRICE_PER_UNIT"]
    .mean()
    .reset_index()
)

plt.figure(figsize=(12, 6))
sns.barplot(data=avg_price, x="LIFESTAGE", y="PRICE_PER_UNIT", hue="PREMIUM_CUSTOMER")
plt.xticks(rotation=45, ha="right")
plt.title("Precio promedio por unidad, por segmento")
plt.tight_layout()
plt.show()
```



13. Prueba de hipótesis (t-test)

Comparamos el precio por unidad pagado por clientes **Mainstream** (jóvenes y de mediana edad, solteros/parejas) contra el pagado por clientes **Budget** y **Premium** del mismo grupo etario, para confirmar si la diferencia observada en el gráfico anterior es estadísticamente significativa.

```
In [88]: # Filtramos las filas del grupo "Mainstream" dentro de las etapas de vida
# jóvenes/mediana edad solteros o en pareja
mainstream_mask = (
    (data["PREMIUM_CUSTOMER"] == "Mainstream") &
    (data["LIFESTAGE"].isin(["YOUNG SINGLES/COUPLES", "MIDAGE SINGLES/COUPLES"]))
)
mainstream_prices = data.loc[mainstream_mask, "PRICE_PER_UNIT"]

# Filtramos las filas de "Budget" o "Premium" dentro de las mismas etapas de vida,
# para comparar contra el grupo Mainstream
other_mask = (
    (data["PREMIUM_CUSTOMER"].isin(["Budget", "Premium"])) &
    (data["LIFESTAGE"].isin(["YOUNG SINGLES/COUPLES", "MIDAGE SINGLES/COUPLES"]))
)
other_prices = data.loc[other_mask, "PRICE_PER_UNIT"]

# ttest_ind realiza una prueba t de dos muestras independientes
# equal_var=False usa la corrección de Welch (no asume varianzas iguales entre grupos)
t_stat, p_value = stats.ttest_ind(mainstream_prices, other_prices, equal_var=False)

# Mostramos el estadístico t y el p-value resultante
print(f"Estadístico t: {t_stat:.4f}")
print(f"p-value: {p_value:.6f}")

# Si p_value < 0.05, la diferencia de precios es estadísticamente significativa
if p_value < 0.05:
    print("La diferencia de precio por unidad SÍ es estadísticamente significativa.")
else:
    print("La diferencia de precio por unidad NO es estadísticamente significativa.")
```

Estadístico t: 37.6244

p-value: 0.000000

La diferencia de precio por unidad SÍ es estadísticamente significativa.

14. Deep dive: afinidad de marca y tamaño de empaque

Analizamos en detalle el segmento **Mainstream - Young Singles/Couples** (uno de los que más contribuye a las ventas) para ver si prefiere marcas o tamaños de empaque particulares, comparado con el resto de la población.

```
In [89]: # Filtramos el segmento objetivo: Mainstream + jóvenes solteros/parejas
target_mask = (
    (data["LIFESTAGE"] == "YOUNG SINGLES/COUPLES") &
```



```

    (data["PREMIUM_CUSTOMER"] == "Mainstream")
)
target = data[target_mask]

# El resto de la población es todo lo que NO está en el segmento objetivo
rest = data[~target_mask]

# .value_counts(normalize=True) da la PROPORCIÓN (no el conteo) de cada marca
# dentro del segmento objetivo, para poder comparar en igualdad de condiciones
target_brand_share = target["BRAND"].value_counts(normalize=True)

# Lo mismo pero calculado sobre el resto de la población
rest_brand_share = rest["BRAND"].value_counts(normalize=True)

# El índice de afinidad es la razón entre ambas proporciones:
# > 1 significa que la marca está sobre-representada en el segmento objetivo
# < 1 significa que está sub-representada
affinity_brand = (target_brand_share / rest_brand_share).sort_values(ascending=False)

# Mostramos las 10 marcas con mayor afinidad hacia el segmento objetivo
affinity_brand.head(10)

```

```

Out[89]: BRAND
TYRRELLS      1.235751
TWISTIES      1.223096
DORITOS       1.210572
TOSTITOS      1.205700
KETTLE        1.193406
PRINGLES      1.181003
COBS          1.137600
INFUZIONI     1.122003
GRNWVES       1.058592
THINS         1.050009
Name: proportion, dtype: float64

```

```

In [90]: # Repetimos el mismo cálculo de afinidad, pero ahora para PACK_SIZE en vez de BRAND
target_pack_share = target["PACK_SIZE"].value_counts(normalize=True)
rest_pack_share = rest["PACK_SIZE"].value_counts(normalize=True)

affinity_pack = (target_pack_share / rest_pack_share).sort_values(ascending=False)

# Tamaños de empaque con mayor afinidad hacia el segmento objetivo
affinity_pack.head(10)

```

```

Out[90]: PACK_SIZE
270      1.272269
380      1.256849
330      1.224477
134      1.181003
210      1.175546
110      1.172431
135      1.136234
250      1.126673
170      0.994629
150      0.964484
Name: proportion, dtype: float64

```

15. Conclusiones (a completar según tus resultados)

- **Marca preferida por el segmento objetivo:** revisa `affinity_brand` para identificar qué marca(s) tienen el índice de afinidad más alto.
- **Tamaño de empaque preferido:** revisa `affinity_pack` de la misma forma.
- **Diferencia de precio:** usa el resultado del t-test para confirmar si el segmento Mainstream paga significativamente más o menos por unidad.

Con esto queda completo el EDA equivalente al template en R, listo para pasar al Task 2 (recomendaciones de estrategia comercial).

15. Conclusiones

Segmento objetivo: Mainstream – Young Singles/Couples

Marca preferida

Marca	Índice de afinidad
Tyrrells	1.24x
Twisties	1.22x
Doritos	1.21x
Tostitos	1.21x
Kettle	1.19x
Pringles	1.18x
Cobs	1.14x
Infuzions	1.12x
Grain Waves	1.06x
Thins	1.05x

Lectura de negocio: el segmento no se concentra en una sola marca, sino en un grupo de 6-8 marcas (21%-24% de sobre-representación) que combinan snacks masivos de sabor fuerte (Doritos, Tostitos) con marcas premium artesanales (Tyrrells, Kettle). Indica un comprador que busca variedad e identidad de marca, no solo precio bajo.

Tamaño de empaque preferido

Tamaño (g)	Índice de afinidad
270	1.27x

Tamaño (g)	Índice de afinidad
380	1.26x
330	1.22x
134	1.18x
210	1.18x
110	1.17x
135	1.14x
250	1.13x
170	0.99x
150	0.96x

Lectura de negocio: preferencia clara por empaques **grandes** (270g–380g), contrario a lo que se esperaría de "jóvenes solteros/parejas" comprando porciones individuales. Los tamaños estándar (150g, 170g) están en o por debajo del promedio del resto de la población. Sugiere consumo para compartir o para varios días, no snacking individual de paso.

Precio por unidad

- **t = 37.6244, p < 0.000001** → diferencia estadísticamente significativa (muy por debajo del umbral $\alpha = 0.05$).
- El signo positivo del estadístico t confirma que el segmento **Mainstream paga más por unidad** que Budget/Premium dentro del mismo grupo etario (Young/Midage Singles & Couples).

Lectura de negocio: este segmento no es sensible al precio en la misma medida que Budget — está dispuesto a pagar un premium por unidad, lo cual es consistente con su preferencia por marcas premium/especializadas (Tyrrells, Kettle) y formatos grandes.

Recomendación estratégica para Julia

El segmento **Mainstream – Young Singles/Couples** es un objetivo de alto valor para la categoría:

1. **Producto:** priorizar Tyrrells, Twisties, Doritos, Tostitos y Kettle en el surtido asignado a este segmento, y en la ubicación destacada de anaquel.
2. **Formato:** enfocar el mix hacia empaques grandes (270g–380g) en lugar de individuales — este segmento compra para compartir/consumo prolongado, no snack de paso.
3. **Precio:** dado que pagan significativamente más por unidad y sin evidencia de alta sensibilidad al precio, hay margen para mantener o incluso testear precios premium en estas combinaciones marca-tamaño sin necesidad de descuentos agresivos, a diferencia

de lo que se recomendaría para el segmento Budget.

4. **Próximo paso sugerido:** cruzar marca × tamaño de forma conjunta (no por separado) para aislar si la afinidad de tamaño viene del tamaño en sí o está arrastrada por la marca que lo distribuye (ej. Pringles en 134g).

Con esto queda cerrado el EDA equivalente al template en R, listo para pasar al Task 2 (recomendaciones de estrategia comercial y presentación a Julia).

In []: